1. Descripción del modelo IA seleccionado y el motivo de su elección:

Para el desarrollo del frontend y la integración con el backend en Laravel, se utilizó ChatGPT como modelo principal de inteligencia artificial.

La elección se basó en evaluacion de otras IA's como Blackbox, DeepSeek, pero ChatGPT ofreció mejores respuestas estructuradas y entendencia en la generación de código

2. Registro de las interacciones con la IA

- Prompt #1- Base técnica del frontend (React + Vite + TypeScript)

El objetivo de este primer prompt fue crear la base técnica del frontend en React usando Vite y TypeScript. Antes de empezar con el diseño, quería tener una estructura clara y organizada para poder trabajar mejor y conectar después con mi API de Laravel.

Solicité a la IA que me generara:
- Comandos de instalación con Vite.
- Dependencias necesarias para un ecommerce moderno.
- Una estructura de carpetas modular y bien organizada.
- Configuración inicial de Axios utilizando variables de entorno (.env).
- Preparación del sistema de autenticación con Bearer Token.

La estructura que me generó estaba bastante bien. Separaba las responsabilidades en carpetas

Quiero que desarrolles el frontend completo de mi proyecto Midori utilizando React(TS) y Vite.

Backend existente:
https://github.com/Inbi-Nav/Midori-Back.git

Primero analiza cuidadosamente el repositorio para comprender:

- Endpoints disponibles
- Sistema de autenticación con Laravel Passport (Bearer Token)
- Roles de usuario (cliente, proveedor, administrador)
- Estructura de datos y modelos
- Requisitos de conexión (base URL, formato de respuestas JSON, etc.)

Después del análisis, genera únicamente la arquitectura base del frontend.

Necesito:

1) Comandos exactos para crear el proyecto con Vite + React + TypeScript
2) Dependencias necesarias para un ecommerce moderno y escalable
3) Estructura de carpetas profesional, modular y limpia
4) Configuración inicial del proyecto
5) Archivos base necesarios (configuración axios, carpeta, etc.)
6) Preparación del sistema de autenticación (estructura, no implementación visual)

En este primer paso NO generes diseño visual, páginas UI ni estilos. Solo quiero la base técnica completamente preparada para empezar el desarrollo.

🚀 1) Crear proyecto con Vite + React + TypeScript

Ejecuta exactamente estos comandos:

```bash
# 1. Crear proyecto
npm create vite@latest midori-frontend -- --template react-ts

# 2. Acceder al proyecto
cd midori-frontend

# 3. Instalar dependencias
npm install
```

🔶 2) Dependencias recomendadas (Ecommerce moderno y escalable)

Instala estas dependencias clave:

```bash
# Axios para HTTP
npm install axios

# Routing
npm install react-router-dom

# Gestión global de estado (elige una o más)
npm install zustand
# o si vas a Redux
npm install @reduxjs/toolkit react-redux

# Forms y validación
npm install react-hook-form yup @hookform/resolvers

# Interfaces tipadas para API
```

Sin embargo, aunque le pasé el enlace del repositorio del backend, la IA no analizo correctamente todas las rutas de la API. Por ejemplo, /auth/login, que no coincidían exactamente con las que yo tenía. Por eso tuve que modificar los endpoints para adaptarlos a los reales de mi backend.

También tuve que ajustar cómo se guardaba y gestionaba el token, ya que la respuesta de Laravel Passport no era exactamente como la IA la genero

http://127.0.0.1:8000/api/auth/refister

El frontend estaba llamando a:
/api/auth/register
En backend las rutas están definidas directamente como: /api/register
Asi que en auth.service los cambie de
axios.post('/auth/register', data)
Por:
axios.post('/register', data)

intentar instalar Axios con, apareció un error relacionado con dependencias. El problema estaba relacionado con un conflicto entre versiones de eslint y eslint-plugin-react-hooks. La versión instalada de eslint no era compatible con el plugin, ya que este solo aceptaba versiones hasta la 8.x, mientras que el proyecto estaba utilizando una versión más reciente. Para solucionarlo modifique desde package.json la version a una anterior.

```
PS C:\Projects\MidorI-Front\midori-frontend> npm install axios
npm error code ERESOLVE
npm error ERESOLVE could not resolve
npm error
npm error While resolving: eslint-plugin-react-hooks@7.0.1
npm error Found: eslint@10.0.0
npm error node_modules/eslint
npm error   peer eslint@"^6.0.0 || ^7.0.0 || >=8.0.0" from @eslint-
npm error   node_modules/@eslint-community/eslint-utils
npm error     @eslint-community/eslint-utils@"^4.8.0" from eslint@1
npm error     @eslint-community/eslint-utils@"^4.7.0" from @typescr
npm error     node_modules/typescript-eslint/node_modules/@typescri
```

- Prompt #2 - Infraestructura de conexión y autenticación

```
Continúa con el desarrollo del frontend del proyecto Midori.

Ahora quiero que generes la página de inicio (home/laning Page).

Contexto:
Midori es una tienda online inspirada en la estética japonesa
minimalista. Quiero un estilo zen, elegante,  moderno, limpia y
ordenada.

La página debe incluir:

- Header con:
  - Logo o texto "Midori"
  - Navegación básica
  - Botones de Login y Registro

- Sección principal con una breve introducción a la marca y su
concepto

- Footer sencillo y elegante

En cuanto al diseño:
- Colores verdes oscuros combinados con blanco y negro
- Espacios amplios y diseño modermo
- Estilo minimalista, moderno y limpio

Requisitos técnicos:
- Usar componentes (Header, Footer etc...)

No implementes todavía la lógica de login ni conexión con la API.
Solo necesito la estructura visual de la página.
```

En este segundo prompt le pedí conectarlo con mi backend en Laravel. Le pedí a la IA que analizara mi repositorio para entender cómo funcionaba la autenticación con Laravel Passport (Bearer Token), que endpoints existían para login y registro, y cómo debían enviarse las peticiones. Lo que quería conseguir era dejar preparada toda la parte técnica para que el login y el registro funcionaran de verdad contra mi API.

La IA generó una configuración bastante completa: interceptor en Axios, estructura para el servicio de autenticación y estado global con Zustand.

5) Estado global de autenticación (con Zustand)

▶ `src/store/auth.store.ts`

TypeScript

```typescript
import create from "zustand";
import { getToken, getUser, saveToken, saveUser, clearAuthData } from "../utils/auth.utils";

interface AuthState {
  token: string | null;
  user: any;
  isAuthenticated: boolean;
  setAuth: (token: string, user: any) => void;
  logout: () => void;
}

export const useAuthStore = create<AuthState>((set) => ({
  token: getToken(),
  user: getUser(),
  isAuthenticated: !!getToken(),
  setAuth: (token, user) => {
    saveToken(token);
    saveUser(user);
    set({ token, user, isAuthenticated: true });
  },
  logout: () => {
    clearAuthData();
    set({ token: null, user: null, isAuthenticated: false });
  },
}));
```

✨ Esto asegura que el estado de autenticación persista aunque se recargue la página.

🚦 6) Protección de rutas

▶ `src/routes/AppRoutes.tsx`

TypeScript

```typescript
mport { Navigate, useRoutes } from "react-router-dom";
mport { useAuthStore } from "../store/auth.store";

xport const AppRoutes = () => {
 const { isAuthenticated, user } = useAuthStore();

 return useRoutes([
   { path: "/", element: <Navigate to="/home" /> },

   // 🔓 Rutas públicas
   { path: "auth/login", element: <div>Login Placeholder</div> },
   { path: "auth/register", element: <div>Register Placeholder</div> },

   // 🔒 Ruta protegida: todos deben estar logueados
   {
     path: "dashboard",
     element: isAuthenticated ? <div>Dashboard</div> : <Navigate to="/auth/login" />,
   },

   // 🔵 Protección por rol
   {
```

- Prompt #3 - Página de inicio (Landing Page)

En este tercer prompt empezar con la parte visual del proyecto. Después de tener preparada la estructura técnica y la conexión con la API, quise comenzar con el diseño de la página principal.

La IA generó una estructura bastante bien organizada, separando los componentes correctamente y aplicando estilos con variables CSS. A nivel técnico estaba bien estructurado y modular.

Sin embargo, el diseño que generó no era exactamente lo que esperaba. Aunque cumplía con lo que pedí a nivel técnico, visualmente se sentía demasiado genérico y poco diferencial. No transmitía realmente esa estética japonesa minimalista que tenía en mente, sino que parecía una landing page estándar con colores verdes.

Ahora crea la página de autenticación del proyecto Midori.

Quiero que esté inspirada en esta referencia:
https://www.hackerrank.com/access-account

La idea es un diseño dividido en dos paneles.

En el lado izquierdo:
- Formulario de login para cliente
- Botón para registrarse si no tiene cuenta
- Opción para recuperar contraseña

En el lado derecho:
- Login exclusivo para administrador

Requisitos funcionales:

- Validación de formularios
- Mostrar los errores que devuelva el backend
- Guardar el token si el login es correcto
- Redirigir según el rol del usuario:
  - Cliente → Página de tienda
  - Administrador → Panel de administración

A nivel visual quiero que mantenga la identidad del proyecto:
Estilo minimalista y moderno, diseño limpio, estructura clara y bien organizada.

No quiero copiar exactamente HackerRank, solo usar la idea del layout dividido como inspiración.

Utiliza componentes reutilizables y mantén el código ordenado.

Estructura de Componentes

Código

```
src/
├─ components/
│  ├─ layout/
│  │   ├─ Header.tsx
│  │   └─ Footer.tsx
│  │
│  └─ home/
│      └─ HeroSection.tsx
│
├─ pages/
│  └─ home/
│      └─ HomePage.tsx
│
├─ styles/
│  └─ global.css
```

- Prompt #4 - Página de Autenticación (Login dividido)

Ahora crea la página de autenticación del proyecto Midori.

Quiero que esté inspirada en esta referencia:
https://www.hackerrank.com/access-account

La idea es un diseño dividido en dos paneles.

En el lado izquierdo:
- Formulario de login para cliente
- Botón para registrarse si no tiene cuenta
- Opción para recuperar contraseña

En el lado derecho:
- Login exclusivo para administrador

Requisitos funcionales:

- Validación de formularios
- Mostrar los errores que devuelva el backend
- Guardar el token si el login es correcto
- Redirigir según el rol del usuario:
  - Cliente → Página de tienda
  - Administrador → Panel de administración

A nivel visual quiero que mantenga la identidad del proyecto:
Estilo minimalista y moderno, diseño limpio, estructura clara y bien organizada.

No quiero copiar exactamente HackerRank, solo usar la idea del layout dividido como inspiración.

Utiliza componentes reutilizables y mantén el código ordenado.

En este cuarto prompt el objetivo fue crear la página de login/registro del proyecto. Quería un diseño inspirado en la estructura de **HackerRank**, dividido en dos paneles: a la izquierda el login de cliente y a la derecha el login exclusivo de administrador.

La IA generó una estructura bastante organizada, separando correctamente los componentes (AuthLayout, ClientLoginForm, AdminLoginForm) A nivel estructural estaba bien planteado.
Errores encontrados y ajustes realizados Durante la implementación encontré algunos errores importantes que tuve que corregir.
Error en la validación del rol

Al hacer login como cliente aparecía el mensaje:

```
const { setAuth } = useAuthStore();
const [serverError, setServerError] = useState("");

const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm<FormData>({
  resolver: yupResolver(schema),
});

const onSubmit = async (data: FormData) => {
  try {
    const response = await login(data);

    const token = response.data.access_token;
    const user = response.data.user;

    if (user.role !== "cliente") {
      setServerError("Este acceso es solo para clientes.");
      return;
    }
```

"Este acceso es solo para clientes."

El problema era que la validación comparaba el rol con "cliente" en lugar de "client". Es decir, había una diferencia entre el valor real que devolvía el backend y el valor que estaba comparando el frontend.

También apareció un error:

Failed to resolve import "../shop.css"

Asi que lo cambie a

import "../../styles/shop.css";

components/auth/AuthLayout.tsx

TypeScript

```typescript
import "./auth.css";

interface Props {
  left: React.ReactNode;
  right: React.ReactNode;
}

export const AuthLayout = ({ left, right }: Props) => {
  return (
    <div className="auth-container">
      <div className="auth-left">{left}</div>
      <div className="auth-right">{right}</div>
    </div>
  );
};
```

- **Prompt #5 - Página principal de tienda (Shop)**

Genera la página principal de la tienda para usuarios autenticados (cliente).

Layout:

- Navegación vertical izquierda:
  - Listado dinámico de todas las categorías obtenidas desde la API
  - Al hacer click → filtra productos por categoría

- Navegación superior:
  - Logo Midori
  - Botón "Provider Request"
  - Cesta
  - Perfil
  - Mis pedidos

- Body principal:
  - Productos organizados por categoría
  - Tarjetas modernas de producto
  - Filtros (precio, categoría, etc.)

Requisitos:

- Obtener productos desde la API real
- Manejar estado de carga
- Manejar errores
- Diseño moderno minimalista

En el quinto prompt le pedi crear la página principal de la tienda para los usuarios autenticados (clientes). Aquí ya no era solo diseño, sino integración real con la API y manejo de estado. La IA generó una estructura bastante completa, separando bien los componentes (ShopLayout, SidebarCategories, ProductCard, Filters, etc.) y creando servicios para consumir la API (product.service, category.service). Esto facilitó mucho la organización del código y lo hizo más escalable. La IA asumía ciertos parámetros en la petición (category_id, min_price, max_price) que tuve que verificar con mi backend para asegurar que coincidieran con los nombres reales de los filtros. La IA genero una buena base estructural, pero fue necesario revisar que todo estuviera alineado con el backend real. Y Otra vez el diseño no considia a lo que queria.

A partir de este prompt el proyecto empezo a ser complejo, ya que la IA empezaba a omitir detalles importantes. Por ejemplo fue el botón **“Provider Request”** en la tienda. Visualmente el botón estaba creado, pero al hacer click no ocurría nada ya que no estaba lamanado a ningun endpoint

- Prompt #6 - Panel de Administración y flujo Cliente → Proveedor

Ahora quiero que desarrolles la parte de administración y el flujo completo de cliente a proveedor en el proyecto Midori.

Te paso los endpoints reales de mi backend (Laravel + Passport) para que utilices exactamente estos y no inventes otros.

AUTH:
POST /register
POST /login
POST /logout

GENERAL (auth required):
GET /categories
GET /products
GET /products/{id}

CLIENT:
POST /orders
GET /orders/me
POST /payments
POST /users/request-provider

PROVIDER:
POST /products
PUT /products/{id}
DELETE /products/{id}
GET /orders
PATCH /orders/{id}/status
POST /categories
PUT /categories/{id}
DELETE /categories/{id}

1) ADMIN PANEL

Debe ser accesible únicamente para usuarios con rol de administrador.

En mi sistema el administrador no crea productos.
Su función es supervisar el sistema.

Funcionalidades del Admin:

- Gestión de usuarios:
  - Ver listado de todos los usuarios
  - Mostrar su rol (cliente, proveedor, admin)
  - Cambiar rol si el backend lo permite

- Supervisión de productos:
  - Ver todos los productos
  - Mostrar qué proveedor los ha creado

- Supervisión de pedidos:
  - Ver listado general de pedidos

- Sección básica de estadísticas si el backend lo permite

- Usar los endpoints reales del backend
- Proteger la ruta para que solo acceda el administrador
- Manejar errores
- Mantener coherencia con el sistema de autenticación existente
- Diseño tipo dashboard moderno, limpio y funcional

En la tienda ya existe un botón "Provider Request".

Quiero que implementes el flujo completo:

- Un cliente puede solicitar convertirse en proveedor.
- La solicitud debe enviarse al backend usando el endpoint correspondiente.
- El administrador puede aceptar o rechazar esa solicitud desde el panel de administración.
- Si el admin acepta, el rol del usuario cambia a proveedor.

3) PANEL DE PROVEEDOR

Cuando un usuario aprobado como proveedor haga login, debe tener acceso a una página específica de proveedor.

Esta página debe incluir:

- Gestión de sus propios productos:
  - Crear producto
  - Editar producto
  - Eliminar producto

- Ver sus categorías disponibles
- Vista organizada y clara tipo dashboard

Requisitos:
- Rutas protegidas por rol (solo proveedor)
- Utilizar los endpoints reales del backend
- Mantener coherencia visual con el resto del proyecto
- Diseño moderno, limpio y bien estructurado

Sea funcional y correctamente separado por roles.

En este prompt le pedi desarrollar toda la parte de administración del sistema y el flujo completo para que un cliente pudiera convertirse en proveedor. Dividí el prompt en tres partes:

Primero panel de administración (solo para rol admin). SegundoFlujo cliente - proveedor. Tercero Panel de proveedor con gestión de productos.

Aunque el prompt era claro y detallado, la IA no generó todos los componentes necesarios en la primera respuesta. Por ejemplo, faltaban partes importantes del panel de administración como la supervisión completa de productos, pedidos y estadísticas. Lo cual tuve que hacer un segundo prompt pidiéndole específicamente que completara los componentes que faltaban usando los endpoints reales:

Al finalizar la implementación del panel de administración y del flujo cliente → proveedor, probé hacer login con todos los tipos de usuario para comprobar que todo funcionara correctamente. En ese momento me di cuenta de que no podía iniciar sesión como proveedor. El sistema mostraba un mensaje indicando que ese acceso era solo para clientes. El problema no estaba en el backend, sino en la lógica del frontend, donde había añadido una validación que bloqueaba el login si el rol no era "client", al revisar vi que la arquitectura no era la correcta. No debería bloquear el login por rol, sino permitir que cualquier usuario autenticado acceda y luego redirigirlo según su rol.

```
setAuth(token, user);

switch (user.role) {
  case "client":
    navigate("/shop");
    break;

  case "provider":
    navigate("/provider");
    break;

  case "admin":
    navigate("/admin");
    break;

  default:
    navigate("/");
}        You, yesterday • feat: fetch
```

Errores detectados al probar el sistema completo

Subida de imagen en productos

Inicialmente el formulario de productos no tenía campo para subir imagen. La IA no había implementado el input de tipo file. Después de añadirlo, apareció otro error: “The image field must not be greater than 2048 kilobytes. Cambie 2048 a 5120

```
public function store(Request $request){
    $request->validate([
        'name' => 'required|string',
        'price' => 'required|numeric',
        'image' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
    ]);

    $imagePath = null;
```

Error detectado: imágenes no visibles en la tienda

Al probar la tienda, las imágenes de los productos no se mostraban en /shop, aunque en el backend estaban correctamente guardadas en /public/images/products. El problema no estaba en Laravel, sino en el frontend.

```
You, 22 hours ago | 1 author (You)
const BASE_URL = import.meta.env.VITE_API_BASE_URL.replace('/api', '');
import { useCartStore } from "../../store/cart.store";

You, yesterday | 1 author (You)
interface Props {
  product: any;
}

      You, 23 hours ago • feat: add dynamic cart and orders management
export const ProductCard = ({ product }: Props) => {
  const { addToCart } = useCartStore();

  return (
    <div className="product-card">
      <div className="product-image">
        <img
          src={`${BASE_URL}/${product.image_url.replace(/^\/+/, '')}`}
          alt={product.name}
        />
```

- Prompt #7 - Carrito y Gestión de Pedidos

Continúa con el desarrollo del frontend de Midori.

Ahora quiero que implementes la funcionalidad de la cesta (carrito de compra) y la gestión de pedidos.

Primero, la cesta debe ser completamente funcional y dinámica.

Debe permitir:

- Agregar productos a la cesta
- Eliminar productos
- Actualizar la cantidad de cada producto
- Mostrar el total calculado automáticamente

La cesta debe actualizarse en tiempo real cuando el usuario interactúe con ella.

También quiero que los productos de la cesta se mantengan aunque el usuario recargue la página

En cuanto al diseño:
- Moderno
- Minimalista
- Limpio

Además, quiero implementar la gestión de pedidos según el rol del usuario:

Para clientes:
- Poder ver el estado de sus pedidos
- Poder cancelar un pedido si todavía no está en proceso de entrega

Para proveedores:
- Poder actualizar el estado de los pedidos (por ejemplo: pendiente, en proceso, enviado, entregado)

Organiza la lógica de forma clara y modular, separando bien la parte de carrito y la parte de pedidos.
Debe integrarse con los endpoints reales del backend.

En este prompt el objetivo fue implementar la funcionalidad completa del carrito de compra y la gestión de pedidos según el rol del usuario. La IA generó una estructura modular usando Zustand para el estado global del carrito y separó correctamente la lógica de carrito y pedidos.

Al probar el codigo que genero la IA daba Error **“Invalid hook call” chunk-RY7GF66K.js?v=09dbe988:904 Uncaught TypeError: Cannot read properties of null (reading 'useCallback') at ProductCard.tsx:3:23**

```
You, yesterday | 1 author (You)
interface Props {
  product: any;
}
  const { addToCart } = useCartStore();


export const ProductCard = ({ product }: Props) => {
```

Solución: mover la llamada dentro del componente:

- Prompt #8 - Página de Orders (Cliente y Proveedor)

En este prompt pedí desarrollar la página de Orders separando responsabilidad según el rol del usuario. El objetivo era que los clientes pudieran ver el estado de sus pedidos utilizando el endpoint real GET /orders/me, y que los proveedores pudieran ver todos los pedidos recibidos mediante GET /orders y actualizar su estado usando PATCH /orders/{id}/status. Todo debía estar integrado con los endpoints reales del backend y organizado de forma modular, separando claramente la lógica para cliente y proveedor.

La IA generó una estructura correcta creando un servicio order.service.ts donde definía las funciones para consumir los endpoints mencionados. También creó una página principal OrdersPage que renderizaba distintos componentes según el rol del usuario autenticado: si el rol era client mostraba ClientOrders, y si era provider mostraba ProviderOrders.

```
Continúa con el desarrollo del frontend de Midori.

Backend:
https://github.com/Inbi-Nav/Midori-Back.git

Revisa el backend para identificar los endpoints relacionados con el
perfil del usuario autenticado y la gestión de contraseña.

Implementa la gestión de perfil para todos los tipos de usuario (cliente,
proveedor y administrador).

Quiero:

1) Página de perfil:
  - Ver datos del usuario autenticado
  - Editar datos personales
  - Guardar cambios usando los endpoints del backend

2) Opción de "Profile Settings" accesible desde la navegación para
todos los usuarios.

3) Funcionalidad de logout disponible para todos los roles.

4) Cambio o recuperación de contraseña:
  - Formulario correspondiente
  - Integración con el endpoint del backend para actualizar contraseña

Organiza la lógica de forma clara y reutilizable para que funcione
correctamente según el rol del usuario.
Mantener coherencia visual con el resto de la aplicación.
```

Error encontrado

```
<section style={{ marginTop: "60px" }}>
  <ProviderOrders />
</section>
```

El proveedor no veía los pedidos en su panel

No estaba integrado en el dashboard

Error: 401 Unauthorized en /api/logout

Durante las pruebas del sistema de autenticación, al realizar logout desde el frontend, el backend respondía con un 401 Unauthorized. Inicialmente parecía un problema de Axios o del interceptor que añade el token, pero tras inspeccionar la petición en DevTools se comprobó que el header Authorization: Bearer sí estaba llegando correctamente al backend. El problema estaba en la configuración de la BD. Laravel Passport estaba utilizando **SQLite**, mientras que se estaban revisando las tablas en **MySQL**. Esto generaba confusión porque los tokens OAuth no existían en la base MySQL. La solución fue migrar completamente el proyecto a MySQL, limpiar la configuración (config:clear, cache:clear), ejecutar migrate:fresh y reinstalar Passport. Tras esta corrección, el logout comenzó a funcionar correctamente.

Error: 400 Bad Request en /orders/{id}/status

Al intentar cambiar el estado de un pedido desde el panel del proveedor, el backend devolvía un **400 Bad Request**.

El backend implementa una máquina de estados:
pending → processing
processing → shipped
shipped → delivered

El problema real estaba en el frontend, donde el <select> utilizaba value="", provocando que se enviara un estado vacío antes del estado seleccionado. Se solucionó usando defaultValue y validando que no se enviara un valor vacío.

Error: Pedido creado con carrito vacío

Inicialmente el backend permitía crear pedidos aunque el carrito estuviera vacío, generando pedidos con total_amount = 0.

Esto suponía un fallo de lógica de negocio. Se solucionó añadiendo validación,required|array|min:1'

Y moviendo toda la lógica de creación al OrderService, donde se valida stock, se calculan subtotales y se usa una transacción para garantizar consistencia.

OTROS ERRORES EN LA GENERACIÓN

Muchas veces no leía bien los datos del backend. En ocasiones no interpreta correctamente los datos proporcionados, por ejemplo en la página de *order overview*. A veces muestra mal el total porque en el frontend se ha puesto **`order.total`**, mientras que en el backend el campo se llama **`total_amount`**

3. Análisis del código generado por la IA

- Separación clara de responsabilidades (Controllers, Services, Models).
- Uso de interceptores de Axios para gestión automática de tokens.
- Validaciones en backend usando `$request->validate()`.
- Uso de transacciones (`DB::transaction`) para garantizar consistencia al crear pedidos.
- Implementación de lógica de negocio en servicios en lugar de controladores.

Sin embargo, fue necesario realizar ajustes:

- Corrección de rutas de importación incorrectas.
- Ajuste del nombre de carpetar.
- Corrección de transiciones de estado inconsistentes entre frontend y backend.
- Eliminación de lógica insegura (precio enviado desde frontend).
- No seguía el diseño que estaba en el prompt

En general, el código generado fue una base sólida, pero requirió validación, adaptación y revisión para garantizar seguridad .

4. Descripción del proceso de conexión entre frontend y backend

La conexión entre frontend (React + TypeScript) y backend (Laravel + Passport) se realizó mediante Axios configurado con una baseURL desde variables de entorno.

Se implementaron los siguientes mecanismos:

- Interceptor de Axios para añadir automáticamente el token Bearer.
- Persistencia del token en localStorage.
- Protección de rutas en frontend mediante control de roles.
- Middleware en Laravel para restringir acceso según rol.
- Validaciones backend para evitar manipulaciones desde el cliente.

Los principales complicaciones eran en errores

- Error 401 por mala configuración de base de datos.
- Inconsistencia en estructura JSON enviada desde frontend.
- Errores 400 por transiciones inválidas en la máquina de estados.
- Confusión entre SQLite y MySQL en entorno local.

5. Reflexión sobre el proceso de aprendizaje

El desarrollo de la app fue una experiencia bastante intensa, una mezcla entre aventura y momentos de agobio. Me ayudó a entender mucho mejor cómo funciona una aplicación completa, tanto por backend como por frontend. Es mi segundo proyecto fullstack, así que ya tenía una idea de por dónde empezar y qué pasos seguir, aunque aun así hubo momentos en los que me perdía o no sabía exactamente qué estaba fallando.

Al ser mi segundo proyecto, entendía bastante mejor lo que estaba haciendo en comparación con el primero. También era la segunda vez que usaba React. Aun así, trabajar con React sigue siendo un reto para mí. Me parece muy ordenado y entendible, me gusta cómo se separan las carpetas y cómo se organiza todo por componentes, servicios y estado, pero a veces la gestión se vuelve compleja.

En cuanto a los errores, es algo que a la vez me gusta y me desespera. Me gusta porque gracias a DevTools puedo ver exactamente qué está pasando, revisar las peticiones, los headers, los estados HTTP y entender dónde está el problema. Pero también lo odio porque muchas veces son errores muy simples o pequeños detalles, como una ruta mal escrita o un valor vacío, que pueden hacerte perder horas hasta encontrar la causa real.

Una cosa que no me gustó nada fue el diseño que generaba la IA. Aunque le daba indicaciones claras sobre el estilo y la estética que quería, el resultado visual muchas veces no tenía nada que ver con lo que había pedido. Eran diseños poco atractivos y bastante básicos, así que en esa parte tuve que modificar bastante el resultado para que encajara con la idea real del proyecto.

6. Contenerización con Docker

CONTENERIZACIÓN BACKEND

Para facilitar la ejecución del proyecto en cualquier entorno y evitar problemas de configuración entre diferentes sistemas, se ha utilizado **Docker** para contenerizar la aplicación.

El proyecto utiliza varios contenedores para separar los distintos servicios del sistema:

- **Frontend**: aplicación desarrollada en React con Vite.
- **Backend**: API desarrollada con Laravel.

Cada uno de estos servicios se ejecuta en un contenedor independiente y se comunican entre sí mediante una red interna gestionada por Docker.

Dockerfile
El backend de la aplicación se construye a partir de un **Dockerfile**, el cual define el entorno para ejecutar la API de Laravel.

La configuración del entorno se compone de tres elementos principales:

- **Script de inicialización del contenedor:** Se ejecuta automáticamente cuando el contenedor del backend se inicia. Su función es preparar el entorno de Laravel antes de arrancar el servidor.

- **Dockerfile**: Define la imagen que se utilizará para ejecutar el backend de la aplicación.

- **docker-compose**: El archivo docker-compose.prod.yml permite ejecutar todos los servicios de la aplicación de forma conjunta.

Docker Compose
Para facilitar la gestión de múltiples contenedores se utiliza **Docker Compose**, que permite definir todos los servicios del sistema en un único archivo de configuración.
Mediante este archivo se configuran:

- Los contenedores de backend y base de datos
- Los volúmenes para persistencia de datos
- Las variables de entorno
- Los puertos de acceso a los servicios

De esta forma, todo el sistema puede iniciarse con un único comando:

CONTENERIZACIÓN FRONTEND

El frontend de la aplicación está desarrollado con **React y Vite** y se ha contenerizado utilizando Docker para facilitar su despliegue.

Dockerfile: utiliza una imagen js para instalar las dependencias del frontend y generar el build de la aplicación mediante npm run build.

Una vez realizados los cambios en el proyecto, estos se suben al repositorio de **GitHub** en la rama main. Posteriormente, desde la plataforma **Railway**, se selecciona el repositorio del frontend y el sistema detecta automáticamente el Dockerfile, realizando el proceso de *build* y *deploy* de forma automática.

Este proyecto podría haberse desplegado sin utilizar Docker, sin embargo decidí utilizarlo como un reto personal para aumentar la dificultad del proyecto y aprender más sobre despliegue de aplicaciones.